

Lekcja 13 – [ZASTĘPSTWO] TERMIN ODDANIA PROJEKTÓW Commits on Feb 23, 2026

Temat: Brak modyfikatorów dostępu

W Pythonie nie ma prawdziwych modyfikatorów dostępu (jak *public*, *private*, *protected* w Javie/C++/C#). Nie istnieją żadne słowa kluczowe do tego celu.

Python używa tylko **konwencji nazewnictwa** (i jednego mechanizmu name mangling). Są dokładnie **trzy poziomy**:

Poziom	Zapis	Co oznacza	Czy naprawdę chroniony?
Public	<code>self.nazwa</code>	Domyślny – każdy może czytać i modyfikować	Nie – całkowicie otwarty
Protected	<code>self._nazwa</code>	Tylko konwencja („nie ruszaj z zewnątrz klasy”)	Nie – dostępny zawsze
Private	<code>self.__nazwa</code>	Name mangling + konwencja	Prawie tak (ale da się obejść)

- **Public** – bez podkreślnika
Wszystko, co nie ma _ na początku, jest publiczne.
- **Protected** – pojedynczy _
Tylko sygnalizuje innym programistom: „to jest wewnętrzne, lepiej nie używaj bezpośrednio”.
Python **nie blokuje** dostępu. W Pythonie **protected** to **tylko konwencja**, nie prawdziwa ochrona.
- **Private** – dwa _
Python automatycznie zmienia nazwę na `_NazwaKlasy_nazwa` (name mangling).
Dzięki temu nie da się przypadkowo odwołać z zewnątrz, ale **nadal można** dostać się przez `_Klasa_nazwa`. To jest **najmocniejsza forma "prywatności"** jaką Python oferuje. Python robi coś specjalnego nazywanego **name mangling** („przekształcanie nazwy”).

Przykład:

```
class Osoba:
    def __init__(self, imie, wiek):
```

```

        self.imie = imie          # ← PUBLICZNA (self.nazwa)
        self._wiek = wiek        # protected (tylko dla porównania)
        self.__haslo = "sekret"  # private (tylko dla porównania)

    def przedstaw_sie(self):
        print(f"Cześć, jestem {self.imie}")

class Raport:

    def generuj_raport(self, osoba: Osoba):

        print(f"RAPORT:")
        print(f"    Imię: {osoba.imie}")          # ← działa zawsze
        # print(osoba._wiek)      # działa, ale nie powinienieś
        # print(osoba.__haslo)    # AttributeError

class Pracownik(Osoba):
    def __init__(self, imie, wiek, stanowisko):
        super().__init__(imie, wiek)
        self.stanowisko = stanowisko

    def pokaz_dane(self):

        print(f"Pracownik: {self.imie}")
        print(f"Stanowisko: {self.stanowisko}")

jan = Osoba("Jan Kowalski", 35)
anna = Pracownik("Anna Nowak", 28, "programista")

raport = Raport()
raport.generuj_raport(jan)
raport.generuj_raport(anna)

anna.pokaz_dane()

print(f"Bezpośrednio: {jan.imie}")

```

Dlaczego nie stosować **Protected** – pojedynczy _

1. Wyobraź sobie, że klasa **Osoba** jest napisana przez kogoś innego (np. bibliotekę, kolegę z zespołu, albo Ciebie sprzed 6 miesięcy).

```

class Osoba:
    def __init__(self, imie, wiek):

```

```

        self.imie = imie
        self._wiek = wiek          # ← PROTECTED (tylko konwencja!)

    def urodziny(self):
        self._wiek += 1

    def pokaz_wiek(self):
        print(f"{self.imie} ma {self._wiek} lat")

```

2. Co się stanie, jeśli Ty użyjesz osoba._wiek z zewnątrz?

```

jan = Osoba("Jan", 30)
print(jan._wiek)      # 30 ← działa
jan._wiek = 100        # zmieniamy bezpośrednio ← ZŁE!

```

To działa... ale jest **niebezpieczne**.

3. Za miesiąc autor klasy Osoba robi update:

```

class Osoba:                                # nowa wersja
    def __init__(self, imie, data_urodzenia):
        self.imie = imie
        self._data_urodzenia = data_urodzenia    # ← nadal protected
        # self._wiek już NIE ISTNIEJE!

    def urodziny(self):
        print(f"🎉 Wszystkiego najlepszego {self.imie}!")

    @property
    def wiek(self):
        today = date.today()
        wiek = today.year - self._data_urodzenia.year
        if (today.month, today.day) < (self._data_urodzenia.month,
self._data_urodzenia.day):
            wiek -= 1
        return wiek

```

Co się stało z kodem, który używał jan._wiek?

```

jan = Osoba("Jan", date(1995, 3, 15))
print(jan._wiek)      # ← AttributeError! Nie ma już takiej zmiennej!
jan._wiek = 50        # ← też błąd!

```

4. TAK POWINNO SIĘ ROBIĆ (najlepsza praktyka 2025)

```

from datetime import date

```

```

class Osoba:
    def __init__(self, imie, data_urodzenia):
        self.imie = imie
        self._data_urodzenia = data_urodzenia    # protected – tylko wewnątrz

    def urodziny(self):
        print(f"🎂 Wszystkiego najlepszego {self.imie}!")

# ===== PUBLICZNY INTERFEJS =====
@property
def wiek(self):                                # getter – tylko odczyt
    """Zwraca aktualny wiek"""
    today = date.today()
    wiek = today.year - self._data_urodzenia.year
    if (today.month, today.day) < (self._data_urodzenia.month,
self._data_urodzenia.day):
        wiek -= 1
    return wiek

@wiek.setter
def wiek(self, nowy_wiek):                    # setter – bezpieczna zmiana
    if nowy_wiek < 0:
        raise ValueError("Wiek nie może być ujemny!")
    # przeliczamy datę urodzenia na podstawie nowego wieku
    today = date.today()
    self._data_urodzenia = date(today.year - nowy_wiek, today.month,
today.day)

jan = Osoba("Jan Kowalski", date(1995, 3, 15))

print(jan.wiek)                                # 30 (lub aktualny wiek) ← OK
jan.wiek = 31                                  # zmienia wiek bezpiecznie ← OK
jan.urodziny()

# jan._data_urodzenia                        # ← działa technicznie, ale NIE RÓB TEGO!

```

Przykład Private – dwa podkreślniki __

```

class KontoBankowe:
    def __init__(self, wlasciciel):
        self.wlasciciel = wlasciciel
        self.__saldo = 5000                # ← PRIVATE (dwa __)

    def wplac(self, kwota):
        self.__saldo += kwota
        print(f"Wpłacono {kwota} zł")

```

```

def wypłac(self, kwota):
    if kwota <= self.__saldo:      # używamy wewnątrz klasy – OK
        self.__saldo -= kwota
        print(f"Wypłacono {kwota} zł")
    else:
        print("Brak środków!")

def pokaz_saldo(self):
    print(f"Saldo: {self.__saldo} zł")    # też wewnątrz klasy

konto = KontoBankowe("Jan Kowalski")

konto.wplac(1000)
konto.pokaz_saldo()          # działa → Saldo: 6000 zł

# Próba bezpośredniego dostępu (to, co większość ludzi próbuje):
print(konto.__saldo)        # ← AttributeError: 'KontoBankowe' object has no
                             # attribute '__saldo'

# Próba zmiany:
# konto.__saldo = 999999    # też nie działa tak, jak myślisz

```

Jak obejść name mangling?

```

# Prawdziwa nazwa po manglingu:
print(konto._KontoBankowe__saldo)    # ← działa! → 6000

# Możemy nawet zmienić:
konto._KontoBankowe__saldo = 100    # teraz saldo = 100 zł
konto.pokaz_saldo()                  # → Saldo: 100 zł

```

Lekcja 14 **TERMIN ODDANIA PROJEKTÓW** Commits on Feb 23, 2026

Temat: Obsługi plików: Otwieranie (with open), czytanie/zapis (read, write), tryby (r, w, a).

Obsługa plików

Python ma bardzo prosty i **bezpieczny sposób pracy z plikami** dzięki wbudowanej funkcji `open()` oraz menedżerowi kontekstu `with`. Dzięki temu nie musisz pamiętać o ręcznym zamykaniu pliku (`close()`), a kod jest czytelniejszy i mniej podatny na błędy.

1. Otwieranie pliku – with open(...)

Zalecany sposób:

```
with open('nazwa_pliku.txt', 'tryb') as plik:
    # tutaj pracujemy z plikiem
    # po wyjściu z bloku with plik jest automatycznie zamykany
```

with

- Automatycznie zamyka plik nawet jeśli wystąpi błąd (wyjątek).
- Kod jest krótszy i czytelniejszy.
- Nie musisz pamiętać o `plik.close()`.

2. Tryby otwierania plików (mode)

'r' – read (domyślny)

Otwiera plik tylko do odczytu. Plik **musi istnieć**, inaczej dostaniesz błąd.

'w' – write

Tworzy nowy plik lub **całkowicie nadpisuje** istniejący.

'a' – append

Dopisuje dane na końcu pliku. Jeśli plik nie istnieje – automatycznie go tworzy.

Dodatkowo często spotykane:

- **'r+'** – odczyt + zapis
- **'w+'** – odczyt + zapis (nadpisuje)
- **'a+'** – odczyt + dopisywanie

Przykład otwierania w różnych trybach:

```
with open('dane.txt', 'r') as plik:    # tylko odczyt
```

```

...

with open('wyniki.txt', 'w') as plik:    # nadpisanie
    ...

with open('log.txt', 'a') as plik:      # dopisywanie
    ...

```

3. Czytanie pliku – metody read(), readline(), readlines()

Przykład - czytanie całego pliku:

```

with open('dane.txt', 'r', encoding='utf-8') as plik:
    # 1. Odczyt całego pliku naraz (najprostsze)
    zawartosc = plik.read()
    print(zawartosc)

    # 2. Odczyt linia po linii (najlepsze dla dużych plików)
    for linia in plik:
        print(linia.strip())          # strip() usuwa \n na końcu

    # 3. Odczyt jako lista linii
    linie = plik.readlines()
    print(linie)

```

Praktyczne przykłady zastosowania:

```

# Przykład 1: Wczytanie konfiguracji
with open('config.txt', 'r', encoding='utf-8') as plik:
    for linia in plik:
        if '=' in linia:
            klucz, wartosc = [x.strip() for x in linia.split('=', 1)]
            print(f"{klucz} = {wartosc}")

# Przykład 2: Wczytanie listy zakupów do listy Pythona
with open('zakupy.txt', 'r', encoding='utf-8') as plik:
    lista_zakupow = [linia.strip() for linia in plik if linia.strip()]
    print(lista_zakupow)

```

4. Zapis pliku – metoda write() i writelines()

```

# Tryb 'w' → nadpisuje plik
with open('wyniki.txt', 'w', encoding='utf-8') as plik:
    plik.write("Wynik obliczeń: 42\n")
    plik.write("Druga linia tekstu\n")

# Można też zapisać wiele linii naraz

```

```

linie = ["Pierwsza\n", "Druga\n", "Trzecia\n"]
plik.writelines(linie)

# Tryb 'a' → dopisuje na końcu (idealny do logów)
import datetime

with open('log.txt', 'a', encoding='utf-8') as plik:
    czas = datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S')
    plik.write(f"[{czas}] Użytkownik załogował się\n")
    plik.write(f"[{czas}] Błąd: coś poszło nie tak\n")

```

Praktyczne przykłady zastosowania:

```

import datetime
# Przykład 1: Zapis listy zakupów (nadpisanie)
zakupy = ["mleko", "chleb", "masło", "jajka"]

with open('lista_zakupow.txt', 'w', encoding='utf-8') as plik:
    for produkt in zakupy:
        plik.write(produkt + '\n')

# Przykład 2: Logowanie błędów (dopisywanie)
try:
    1 / 0
except Exception as e:
    with open('errors.log', 'a', encoding='utf-8') as plik:
        plik.write(f"[{datetime.datetime.now()}] BŁĄD: {e}\n")

```